

操作系统

第二章 进程管理

第二部分

进程状态、状态转换 与 调度 (1)

同济大学计算机系



Part 1、多道系统中的进程状态

- 为什么多道？
- 进程调度状态
- 进程状态转换图

进程的执行

- 输入阶段、计算阶段、输出阶段 交错
- 输入阶段，使用输入设备，将新数据读入进程图像（数据段、堆栈段）
 - 读磁盘文件、网络链接（socket）、接收其它进程传来的数据
- 计算阶段，使用CPU，处理读入新数据，生成运行结果
- 输出阶段，使用输出设备，将运行结果存盘 或者 送给其它进程处理



输入输出设备

- 输入设备
 - 磁盘、网络、终端（键盘）
- 计算阶段，处理读入新数据，生成运行结果
- 输出阶段，将运行结果存盘 或者 送给其它进程处理
 - 磁盘、网络、终端（屏幕）

进程 与 外设的同步



处理数据



数据提供者 & 传输通道

事实:

- 1、输入操作没完成，没有可供处理的新数据，进程没法运行
- 2、输出操作没完成，生成的新数据没地方存放，进程也没办法运行

输入、输出操作很慢，比CPU访问内存慢若干数量级。所以，绝大多数情况下，是运算(CPU) 等 输入输出(外设)。



0、为什么多道?

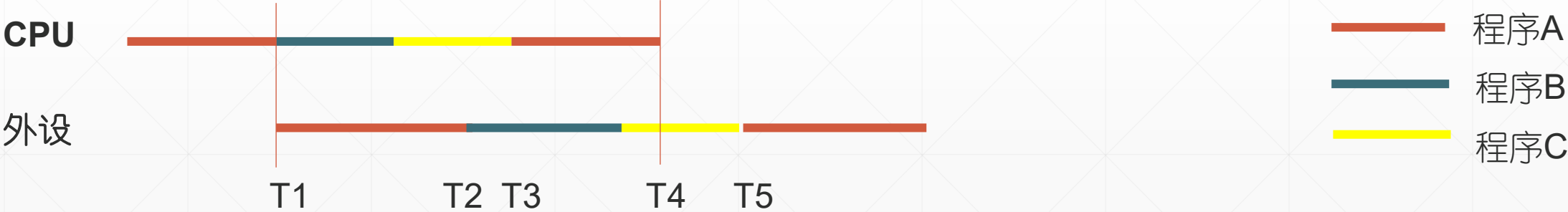


单道系统和多道系统的运行情况

单道系统一次只能跑1个程序，这个程序运行完毕，系统才能装入、运行其它程序。



多道系统可以同时跑很多程序。正在运行的程序IO时，让出CPU给其它进程运算。



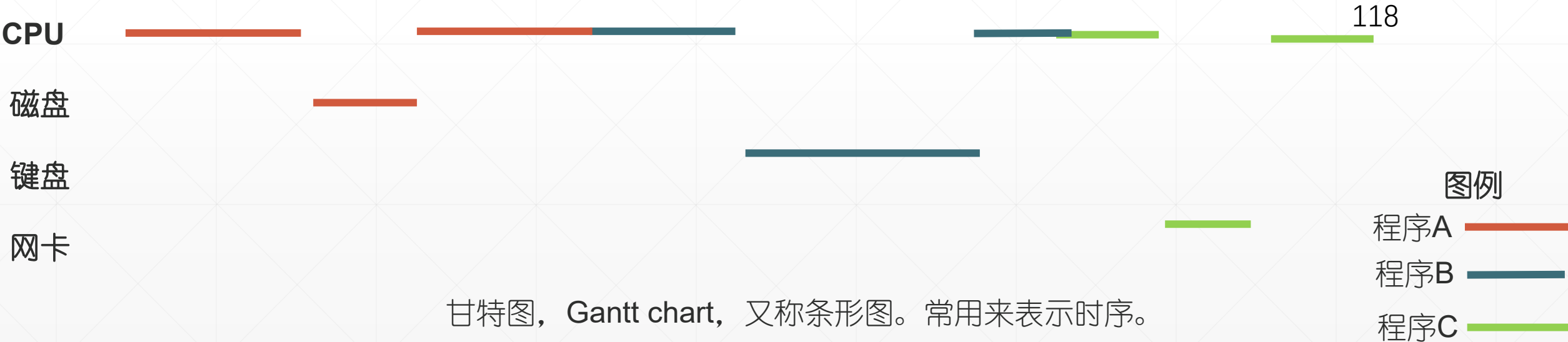
[T1, T4] 外设和CPU全忙，跑完这3个程序，总耗时减少 $T4 - T1$

使用同一个外设



例1：某单道系统， 需要执行3个程序。先来先服务。
画时序图。计算完成所有计算任务的总耗时（118ms）。

程序	进入系统的时刻	第一轮计算时间	外设IO时间 (使用不同的外设)	第二轮计算时间 (之后结束)
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



甘特图， Gantt chart， 又称条形图。常用来表示时序。

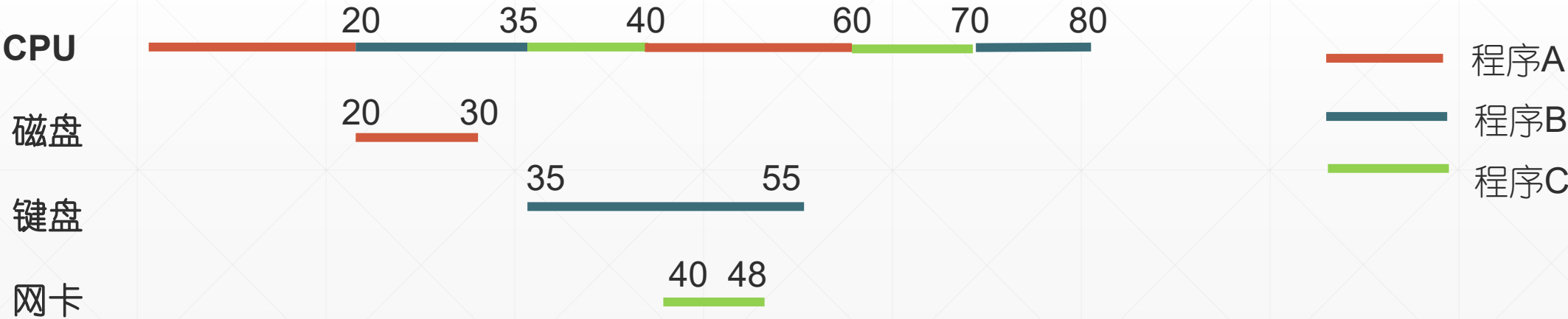


规则： 启动IO，程序放弃CPU。
IO完成后，程序可以使用CPU继续运行

例2：多道程序同时运行（先来先服务）

■ 80ms

程序	进入系统的时间	第一轮计算时间	外设IO时间	第二轮计算时间（之后结束）
A	0	20	10（读磁盘）	20
B	10	15	20（读键盘）	10
C	20	5	8（读网卡）	10



总结

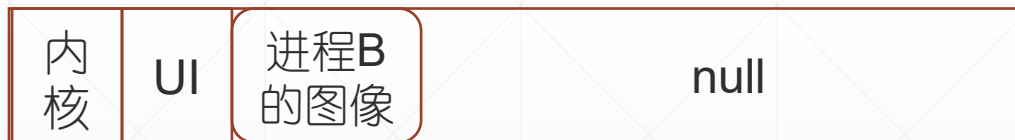
- 单道系统的缺点：↵
 - 无论何时，至多一块芯片在工作。IO、计算不能并行。↵
 - 系统资源利用率低。吞吐率低。↵
 - 单道系统的优点：↵
 - 程序独占计算机系统。从开始运行至运行结束，耗时最短。
↵
 - 多道系统的优点：↵
 - 系统（所有资源）利用率高，吞吐率高。↵
- 优点，来自计算、IO 并行的设计。↵

最后。内存中存放好多程序供 CPU 选择是有好处的。CPU 空闲时，找到没在 IO 可供执行的程序，概率高。有利于保持 CPU 忙碌。另一方面，忙碌的 CPU 会执行更多的应用程序，发出更多的 IO 命令，从而保持 IO 设备忙碌。↵

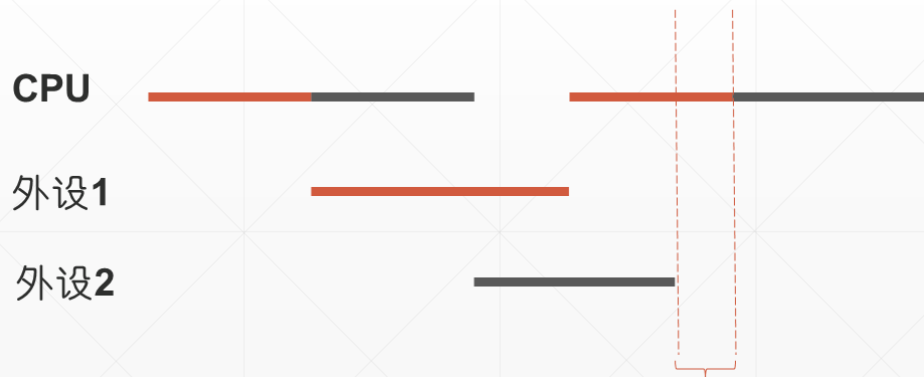
内存装的程序越多，越有可能让所有设备保持忙碌，从而提高系统的吞吐量。这里有一个度，需要对系统中并发的进程总数进行控制。↵

多道系统的内存中并发很多进程图像

单道



多道



管理并发进程，多道系统必需具备以下功能：

1. 能够运行可重定位的程序（编译好的程序，不出现物理地址），确保无论把程序放在哪里，它都能跑起来
2. 为程序分配内存 空间分配
3. 保证运行中的进程不会破坏操作系统，不会访问分配给其它进程的内存空间 隔离
4. 安排多个程序的执行的先后次序 调度
5. 设置CPU，执行新选中进程 进程切换（完成一半，为新选中的进程建立地址映射关系）
6. 保证任何用户未经授权不得访问其它用户的软件资源（程序和数据文件） 安全（权限检测和 身份认证）
7. 实现应用系统的控制逻辑，比如 sleep(10), fork, pthread 进程控制



用来提升系统性能，改善用户体验的其它功能：中断、DMA、缓存

典型的多道操作系统：Windows, Unix, Linux

1、任意时刻，一颗CPU，只能运行一个进程

- MMU引用的页表，反映该进程虚空间布局 & 地址映射关系
- 执行单元正在执行该进程的一个子程序。
 - 通用寄存器是这个子程序的执行现场：
 - EIP，下条指令
 - ESP、EBP定位这个子程序的栈帧
 - EAX，存放这个子程序的返回值
 - EFLAGS中的bit，标识这个程序的运行情况
 - ZF为1，算数运算结果为0
 - IF为1，当前正在执行的代码，开中断运行。
0，当前正在执行的代码，关中断运行，CPU不响应中断。
 -
 - CS，最后2位是该进程的执行状态
 - 00，核心态运行。正在执行系统调用。。。
 - 11，用户态运行。正在执行应用程序

2、进程调度状态

- CPU正在执行的进程，运行态。
- CPU不能执行的进程，阻塞态。
- 等待执行的进程，就绪态。

每个阻塞进程，有确切的阻塞原因：

- 没有可供处理的新数据
 - 磁盘IO操作没完成
 - 等待键盘输入数据
 - 等待网络包

} & 数据缓存块
- 有数据，但不能处理
 - 数据被别的进程锁住了

& 锁变量
- 定时器没到期
 - sleep(10)，之后的代码要10s之后才能跑

& 定时器变量

每个阻塞进程，阻塞原因表示为：

- 0，没有阻塞
- 非0，是一个内核变量的首地址

案例：Unix V6++进程的调度状态

```
class Process
{
public:
    enum ProcessState
    {
        SNULL      = 0,    // 空元素
        SSLEEP     = 1,    // 阻塞，高优先权睡眠
        SWAIT      = 2,    // 阻塞，低优先权睡眠
        SRUN       = 3,    // 运行 或 就绪
        SIDL       = 4,    // 进程创建中，不接受调度
        SZOMB      = 5,    // 进程已终止，仅剩PCB
        SSTOP      = 6,    // 被debug的进程
    };

public:
    .....
    ProcessState p_stat;    // 进程调度状态
    unsigned long p_wchan;  // 睡眠原因
    .....
}
```

读写磁盘数据，阻塞

其它原因，阻塞

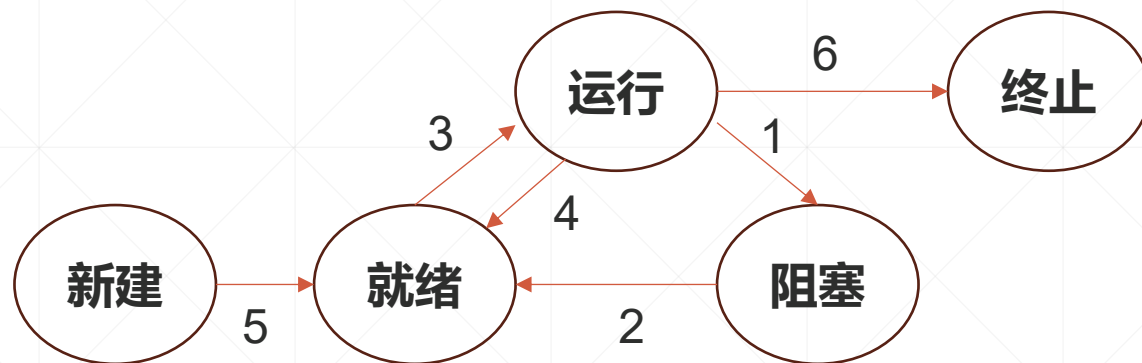


例 2

单核计算机系统，并发20个进程。
运行态进程最多几个？最少几个？
阻塞态进程，最多几个？最少几个？
就绪态进程最多几个？最少几个？

4核计算机系统，并发20个进程。
运行态进程最多几个？最少几个？
阻塞态进程，最多几个？最少几个？
就绪态进程最多几个？最少几个？

3、进程状态变迁



- 1、入睡（阻塞）：进程无法前进主动放弃CPU
- 2、唤醒：IO完成.....进程恢复可执行状态。
- 3、被调度：被选中成为现运行进程。
- 4、被抢占：系统中出现优先级更高的进程。
- 5、进程图像创建完毕，就绪态
- 6、应用程序执行完毕或被杀死

入睡原因

- 1) 启动IO操作
- 2) sleep(10), 启动定时器
- 3) 内存中没有新数据
- 4) 进程访问的数据被其它进程锁住了
- 5) 排队等待资源

唤醒事件

- 1) IO完成
- 2) 定时器到期, 10s后进程就绪
- 3) 新数据就绪 (前驱任务完成 或 IO操作完成)
- 4) 对方数据访问完毕, 解锁
- 5) 获得使用权